AI4009 Generative AI — Assignment 1: Latent Spaces and Why GANs Are Hard to Train

Roll **25L-8006** • seed **8006** • held-out digit **6** • my six digits **{1, 2, 5, 6, 8, 9}** • my attributes **Smiling + Eyeglasses**

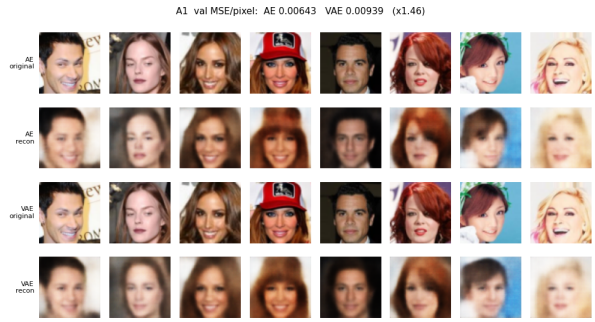


Figure 1: **A1.** Originals over reconstructions, 8 columns, for each model. The AE keeps the red cap, the background lettering and the green hairband; the VAE dissolves them.

1 A — An AE's latent space is not a space

1.1 A1 Train both (CelebA, $d = 64$, 20 epochs)

One encoder–decoder used twice; the models differ only in the reparameterisation, the two heads and the KL term. Reconstruction is summed over pixels to match the KL summed over the 64 latent dimensions; a per-pixel mean would shrink it 12288× and the KL would dominate into posterior collapse. Reported MSE is per pixel.

	train MSE	val MSE
AE	0.006175	**0.006432**
VAE	0.009394	**0.009387**

The VAE is blurrier by +0.002955 MSE/pixel, i.e. 1.459× the AE's error, or 21.92 dB vs 20.27 dB PSNR — a gap of 1.64 dB. Its KL settled at 57.51 nats (≈ 83 bits through a 64-d bottleneck) by epoch 5 and held there, so no posterior collapse. A second difference: the AE ends at 0.006175 train / 0.006432 val with a bouncing validation curve, the VAE at 0.009394 / 0.009387 — identical and monotone. The KL is also acting as a regulariser.

1.2 A2 What the VAE can do: interpolate

Eleven frames, interpolating the *means* μ rather than samples from q. **Every intermediate frame is a plausible face** — none degenerates into non-face structure. **Both attributes change gradually, neither jumps.** For Smiling the frame-to-frame MSE rises monotonically (0.000615 → 0.001366) with no interior peak. For Eyeglasses it peaks mid-strip at $t = 0.4 \to 0.5$ (0.002529), which looks like a jump — but decomposing by image region shows it is not: the eye band (rows 20–36) peaks at $t = 0.1 \to 0.2$ and decays monotonically thereafter, at 1.36× its median, while the mouth band on the Smiling strip is flat at 1.06×. The mid-strip peak is identity and background morphing, not the glasses. So the glasses region changes early and concentratedly while the smile changes at a constant rate; whole-image MSE cannot separate attribute from identity drift.

1.3 A3 What the AE cannot do: generate

The same 16 codes $z \sim \mathcal{N}(0, I)$ through both decoders (Fig. 3): the AE returns brown and grey mush, the VAE returns faces. The third panel decodes the AE's *own* encoder outputs and is the sharpest of the three — the decoder is not broken.

The measurement explains it. For a 64-d standard normal $\mathbb{E}\|z\| \approx 8$; measured on 2000 validation images:

	mean $\|z\|$	mean per-dim std
AE codes	32.06	3.847
VAE μ	6.73	0.837
$\mathcal{N}(0, I)$	7.97	1.000

The AE's codes sit four times further from the origin than any prior draw and the two histograms in Fig. 4 **do not overlap at all**: no prior sample lands in the radial band the decoder was trained on, so it is extrapolating. The VAE's μ are slightly under-dispersed (std 0.837), which is correct rather than a fault: $\text{std}(\mu)^2 + \mathbb{E}[\sigma^2] \approx 1$ implies $\mathbb{E}[\sigma^2] \approx 0.30$, i.e. an encoder genuinely using its noise, consistent with 57.5 nats of KL.

2 B — What the AE is actually good for

2.1 B1 Denoising

Noisy input $\tilde{x} = \text{clip}(x + 0.3\varepsilon, 0, 1)$, clean target, 10 epochs. Mean test PSNR: **13.32 dB noisy → 19.80 dB denoised**, a gain of **6.47 dB**. Test noise is drawn from a fixed generator so the number is reproducible across epochs.

Why can a model that cannot generate a face remove noise from one? Because denoising only requires projecting a corrupted input back onto the data manifold, and the noise is off-manifold, so a narrow bottleneck discards it almost by construction. Generating requires something strictly stronger — a distribution over codes that can be *sampled* — and A3 shows the AE has no such thing.

2.2 B2 The held-out digit as an anomaly

An AE trained on MNIST with every 6 removed, scored by per-image reconstruction MSE. ROC-AUC computed from the Mann–Whitney rank statistic (not imported): **AUC = 0.883**; mean error 0.00837 on seen digits vs **0.01846** on the held-out 6, a ratio of 2.21×. The per-digit means (`figures/B2_anomaly.png`) put 8 (0.01199) and 2 (0.01159) next, which is the expected confusion given a 6's geometry. Same model, same bottleneck: useless as a generator, useful as a detector.

3 C — How a GAN fails

3.1 C1 Watch the gradient die

Saturating loss $L_G = \log(1 - D(G(z)))$, five D steps per G step, $\text{lr}_D = 8\text{e}{-4}$ against $\text{lr}_G = 2\text{e}{-4}$.

> $\|\nabla_{\theta_G}\| = \mathbf{6.2805 \times 10^{-5}}$ at iteration 100; it first fell below 1% of that (6.2805×10^{-7}) at iteration **345**.

A rolling-median (window 51) version of the same rule gives iteration **761**, so 345 is the first crossing and 761 the sustained one. Both are in `logs/C1_saturating.csv`. Note that the iteration-100 reference is itself already 135× below the peak of 8.49×10^{-3} at iteration 1 — measured from the true peak, the gradient was below 1% by iteration **32**. By the end $D(x) = 1.000$, $D(G(z)) = 9.8 \times 10^{-11}$ and $\|\nabla_{\theta_G}\| = 2.0 \times 10^{-10}$, a fall of 4.2×10^7 from peak. The samples never leave their initialisation (Fig. 8, left).

3.2 C2 Fix it with one line

Identical seed, architecture and D schedule; only $L_G = -\log D(G(z))$.

median $\|\nabla_{\theta_G}\|$, last 500 it.	C1	C2
	6.24×10^{-10}	3.265

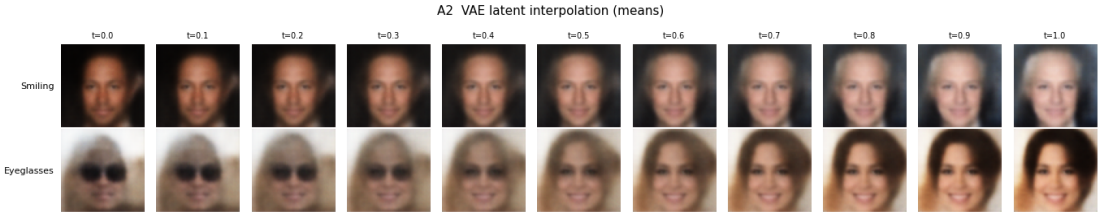


Figure 2: **A2.** VAE latent interpolation between encoded means, $t = 0.0 \ldots 1.0$. Top: Smiling $+1 \to -1$. Bottom: Eyeglasses $+1 \to -1$.

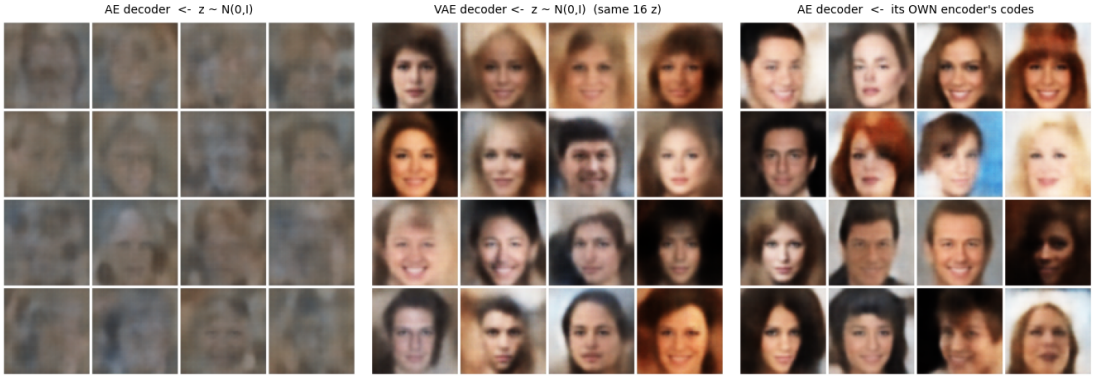


Figure 3: **A3.** The same 16 codes through both decoders, and (right) the AE decoder on codes its own encoder produced.

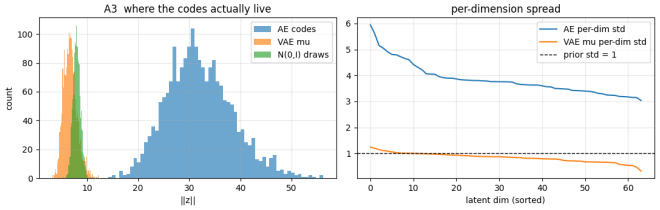


Figure 4: **A3.** Where the codes actually live. The AE cloud (15–55) and the prior draws (5–11) are disjoint.

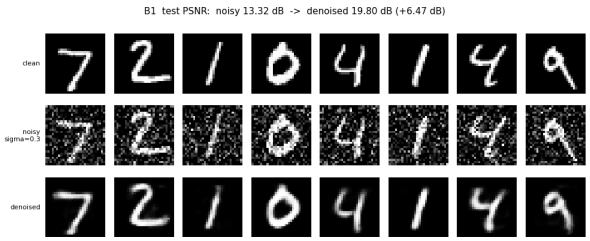


Figure 5: **B1.** Clean / noisy / denoised.

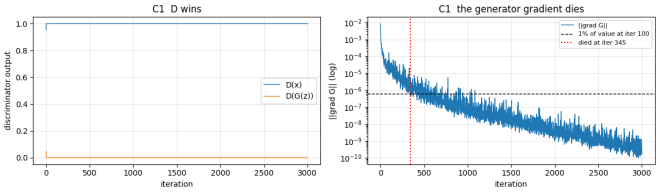


Figure 6: **C1.** $D(x)$ and $D(G(z))$, and the generator gradient norm on a log axis with the death iteration marked.

A ratio of 5.2×10^9. Both losses share a fixed point, since both are minimised as $D(G(z)) \to 1$, but their gradients differ by a factor of $D(G(z))^2$ in the regime that matters. The saturating loss carries a prefactor $\propto D(G(z))/(1-D(G(z)))$, so when a strong D drives $D(G(z))$ to 10^{-10} the update vanishes exactly when the generator is worst and most needs signal. The non-saturating loss scales as $1/D(G(z))$, which *grows* in that regime: under the identical schedule C2's gradient held at 2–3.4 and rose during training, with $D(x) = 0.894$, $D(G(z)) = 0.030$ at the end. D still wins; the difference is that G keeps receiving a usable gradient while losing.

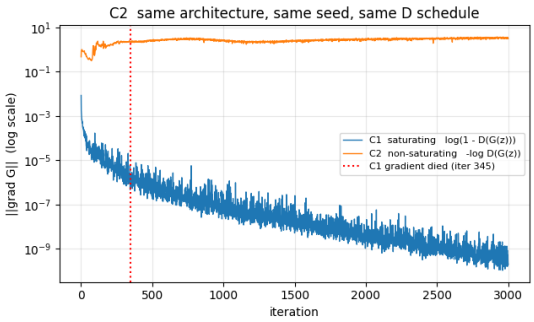


Figure 7: **C2.** Both gradient-norm curves, log axis, same seed and schedule.

3.3 C3 Make it collapse, then measure it

Recipe as given: z-dim 2, no BatchNorm in G, five G steps per D step, $\mathrm{lr}_G = 2\mathrm{e}{-3}$ vs $\mathrm{lr}_D = 2\mathrm{e}{-4}$. The result is a **total** collapse: of 5000 classified samples, **99.94% are digit 9**.

modes covered	reverse-KL	sample std
1 / 6	**1.7867**	0.0129

(Classifier test accuracy 0.9895 on the six digits.) One honest caveat about the stopping criterion: the samples did *not* stop changing. The MSE between consecutive fixed-noise grids fell only from 0.926 to 0.804 over 1500 iterations (`figures/C3_movement.png`). Every z maps to essentially the same image at any given iteration — that is what the 0.0129 diversity says — but that single image keeps drifting from iteration to iteration. The collapse is in class coverage, not in pixel stasis. D meanwhile sat at chance ($D(x) = 0.494$, $D(G(z)) = 0.464$), which is the mechanism: G races ahead of a stale D and parks every z on the one sample that fools it.

4 D — A GAN that actually works

4.1 D1 DCGAN

Strided convolutions, BatchNorm in both networks, LeakyReLU(0.2) in D, ReLU in G, tanh output, Adam(2e−4, $\beta_1 = 0.5$), non-saturating loss, 1:1 steps, 5000 iterations. Result: **6/6 modes, reverse-KL 0.0182**, histogram [0.126, 0.203, 0.191, 0.126, 0.163, 0.191] against a uniform target of 0.167. At the end $D(x) = 0.578$, $D(G(z)) = 0.385$, $d_{\text{loss}} = 1.110$

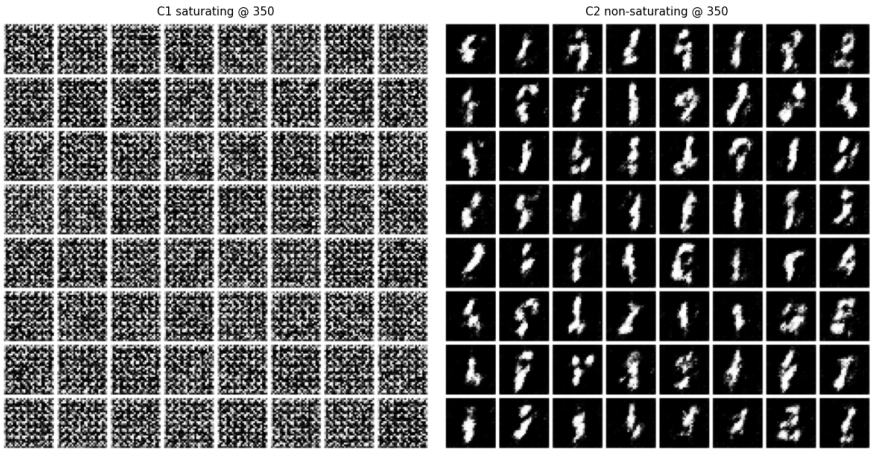


Figure 8: **C2.** The two generators at the same iteration (345/350). One line of loss separates them.

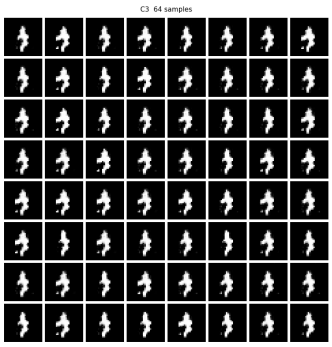


Figure 9: **C3.** 64 samples. One mode, one digit.

— a discriminator that genuinely cannot separate the two distributions.

4.2 D2 One stabiliser, honestly reported

Minibatch discrimination (Salimans et al.), written out rather than imported, placed in the *discriminator*. A unit test confirms it responds to batch similarity: an identical batch gives a feature of 7.000, a diverse batch 0.228.

It did not help. IS improved marginally (4.206 vs 4.119) but both distribution-level measures got worse: reverse-KL 0.0665 vs 0.0182, FID 38.75 vs 27.48, with mode coverage unchanged at 6/6. The reason is in the training logs: the extra batch-similarity channel strengthened an already dominant discriminator ($D(x) = 0.998$ vs 0.578, $d_{\text{loss}} = 0.085$ vs 1.110, $g_{\text{loss}} = 7.0$ vs 0.86). D1 had not collapsed, so there was no mode collapse left to fix; the only effect of the stabiliser was to push the game further from equilibrium.

5 E — Measuring it: IS and FID

Both metrics implemented from the formulas on the six-digit classifier: its softmax gives $p(y|x)$, its 128-d penultimate layer gives the FID features. No `pytorch-fid`, `clean-fid` or scikit-learn was used at any point, including for cross-checking. Validation is by cases with known answers: IS of a confident, uniformly distributed predictor $= K = 6$; IS of a maximally unsure predictor $= 1$; FID$(X,X) = 0$; and a mean shift of 3 across 32 dimensions returns 288.2 against an analytic 288.

model	modes	rev-KL	IS	FID
real data	–	–	5.698 ± 0.031	0.32
C1 saturating	2/6	1.6288	1.093 ± 0.004	624.31
C3 collapsed	1/6	1.7867	1.042 ± 0.002	670.66
D1 DCGAN	6/6	0.0182	4.119 ± 0.077	**27.48**
D2 + minibatch	6/6	0.0665	$\mathbf{4.206 \pm 0.066}$	38.75

IS is over 10 splits of 5000 samples; FID is against the real training set (10 000 images).

5.1 E2 The matrix square root

$\Sigma_r\Sigma_g$ is a product of two PSD matrices, so its eigenvalues are real and non-negative and the true square root is real. An imaginary component can nevertheless appear from rounding in the Schur decomposition that `scipy.linalg.sqrtm` uses internally, which is why one takes `.real`. In this run the conditioning was visibly poor — SciPy raised `LinAlgWarning: Matrix is singular` on every call, since a 128-d covariance is estimated from 5000 samples — but the returned array was purely real: the recorded imaginary magnitude was **exactly 0.0** for all five sets, so `.real` discarded nothing.

5.2 E3 Two sanity checks

The floor. FID between two disjoint halves of the real test set (3000 each) is **0.568**, not zero. It measures finite-sample estimation error: μ and Σ computed from 3000 samples differ from those of another 3000 samples drawn from the same distribution, so 0.568 is the noise level below which no FID difference is meaningful.

Monotonicity. FID against the real set for Gaussian noise $\sigma = 0, 0.1, 0.25, 0.5, 1.0$: 0.375, 7.23, 60.14, 265.19, 566.70 — monotone (Fig. 12).

5.3 E3(c) What each metric is blind to

My collapsed model scored IS 1.042 and FID 670.66 against D1's 4.119 and 27.48. IS never looks at the real data at all — it only asks whether each sample is confidently *something* and whether the set spreads across labels — so it cannot know that four of my six digits are missing, and it is bounded above by the number of classes actually covered, which is why a 1-mode generator is capped near 1 regardless of how clean its 9s are. FID does compare against real data, but only through a mean and a covariance in a 128-d feature space, so it cannot say *which* digits vanished and would score two entirely different failures identically if their first two moments matched. The sharpest evidence is that my C1 model, which emits pure salt-and-pepper noise, beats the collapsed model on *both* metrics (IS 1.093 vs 1.042, FID 624.31 vs 670.66) — neither metric can see that one of these generators produces clean, recognisable digits and the other produces no images at all. Both numbers are also hundreds of times above my 0.568 floor, so the ranking is not noise; it is what these statistics genuinely measure.

6 What surprised me

1. A prior draw is *closer* to the AE's codes than the codes are to each other. I expected the opposite. Nearest-neighbour distance from $\mathcal{N}(0,I)$ draws to AE codes is **16.09**, while the distance between AE codes themselves is **25.45** (Fig. 4). This is not a contradiction of A3, it is 64-d geometry: the AE's per-

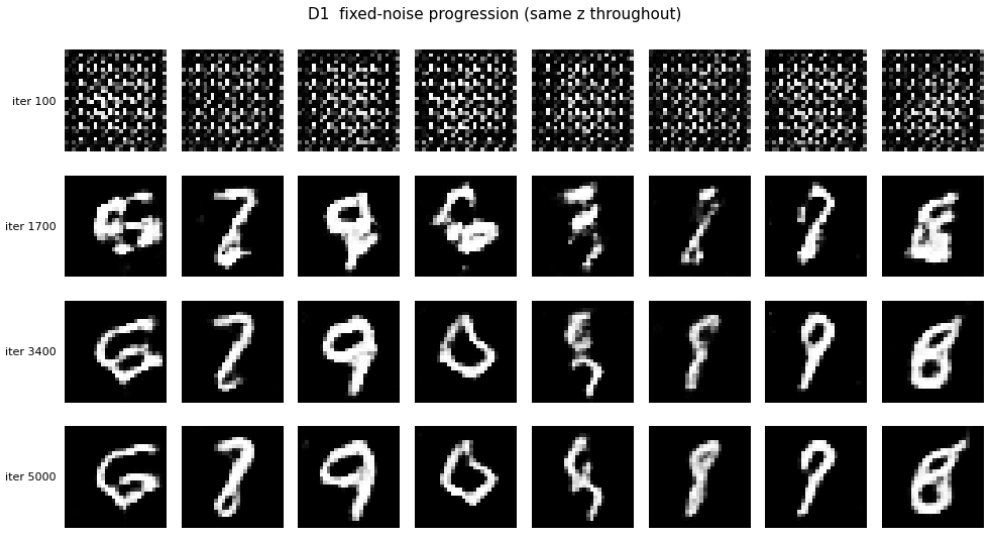


Figure 10: **D1.** The same fixed z at four points in training: noise, then structure, then digits.

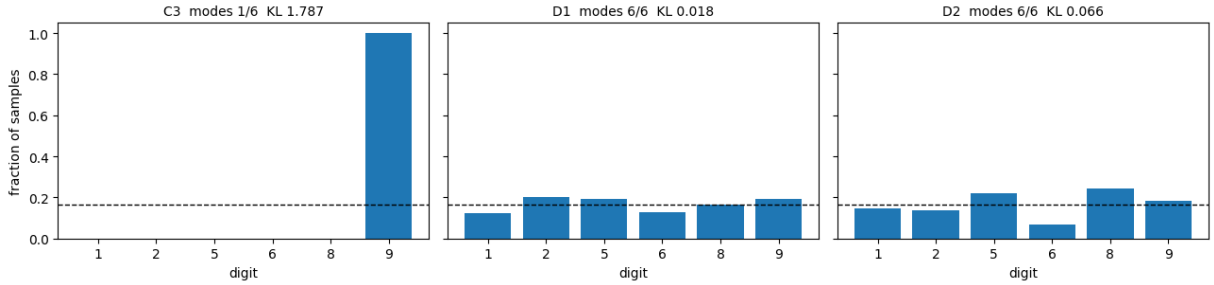


Figure 11: **D1/D2.** Class histograms for all three models, dashed line = uniform target.

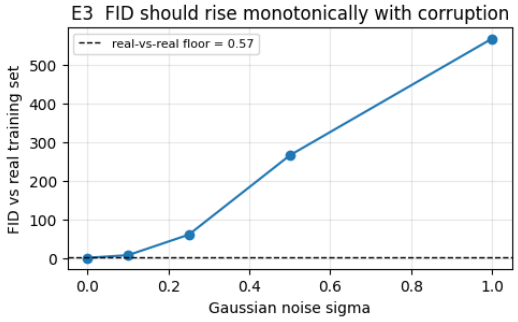


Figure 12: **E3.** FID rises monotonically with corruption.

dimension mean is -0.175, so the codes form a diffuse *shell* of radius ≈ 32 whose points are enormously far apart, and a prior draw lands near the origin — the hole in the middle. Being central makes you moderately close to everything while still sitting where no encoder ever placed a code. Euclidean nearest-neighbour distance is simply the wrong instrument for "is this point inside the data distribution" when the data lives on a shell; the radial histogram is the right one.

2. Pure noise scores better than mode collapse on both metrics. C1's generator never left its initialisation and emits salt-and-pepper (Fig. 8, left), yet it beats the collapsed C3 model on IS (1.093 vs 1.042) *and* on FID (624.31 vs 670.66). The lecture framing — IS rewards confident, diverse output, FID punishes distribution mismatch — would have predicted C3 ahead on both. What actually happens is that a diffuse noise cloud overlaps the real feature statistics slightly better than a tight cluster sitting in the wrong place, and the classifier assigns noise to a couple of classes with moderate confidence.

7 What I could not get to work

Isolating the attribute in A2. Three attempts all measured the identity morph instead: whole-image frame MSE peaked mid-strip; band-restricted MSE put the eye region's peak at $t = 0.1$–0.2; band brightness was dominated by skin and hair and *fell* across the Eyeglasses strip even as the glasses vanished. The endpoints differ in every respect, so no global statistic on an image pair separates them. The right tool is an attribute vector (difference of mean μ with and without the attribute, applied to one fixed identity), which A2 did not ask for and I did not run.

The C3 stopping criterion. "Train until the samples stop changing" never became true: consecutive fixed-noise MSE fell only 0.926→0.804 over 1500 iterations. With z-dim 2 and no BatchNorm the generator keeps wandering; what converged was class coverage, not the images. I report the class collapse as the result and the movement curve as the caveat rather than claiming a convergence I did not observe.

The C1 reference point. With this D schedule the gradient had already fallen 135× from its peak by iteration 100, so the handout's threshold describes a late stage of a collapse mostly over by iteration 32. I report both numbers.

The stabiliser. D2 was worse on two of three measures. I did not tune it further: with D1 already at 6/6 and reverse-KL 0.018 there was no failure left to fix, so a better-tuned minibatch discriminator would at best match the baseline.

8 Declaration of LLM use

I used Claude (Anthropic) for: scaffolding the Kaggle CelebA loader and the training loops around the provided starter; explaining the ELBO scaling issue (summed vs per-pixel reconstruction); debugging an `IndexError` in my A3 nearest-neighbour baseline and an `IndentationError` in the FID function; discussing how to read the results, including the three failed A2 measurements and the C1-vs-C3 inversion; and drafting this report's structure from my own numbers. Every run, number and figure here is from my own execution of my own notebook under seed 8006, and I can derive any part of it.